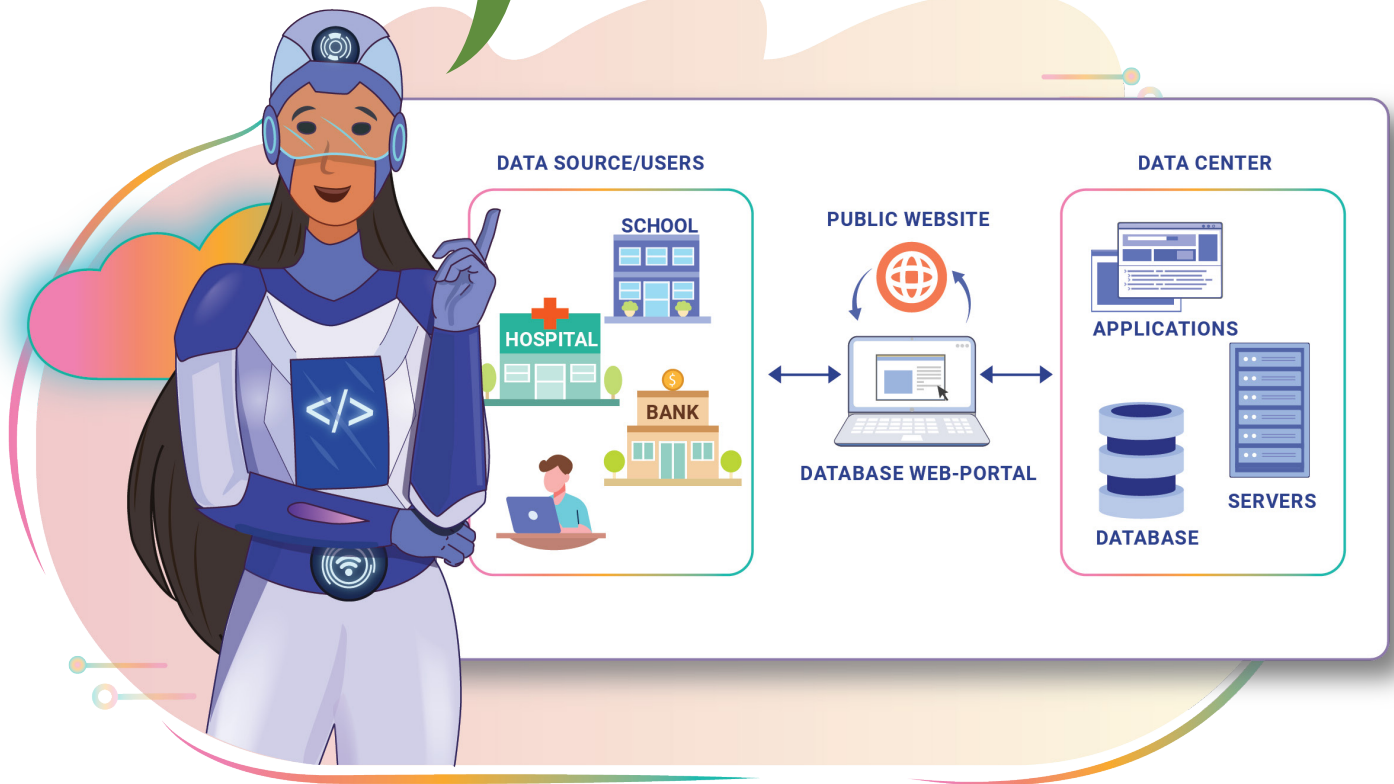




Introduction to MySQL

07

A DBMS is crucial for efficiently managing and organizing vast amounts of data, ensuring data integrity, and enabling easy access and retrieval, thus facilitating effective decision-making and business operations.



Introduction

In this lesson, we are going to get familiar with databases. We will learn what databases are, how to store data, and explore other related functionalities.

Before we get started with Database, let us understand what Data is.

What is Data?

Data can be simply defined as information related to an object, available in different formats, such as integers, media, texts, and so on. Examples of data can include prices, costs, names of products, dates, times, locations of events, your name, address, age, etc.

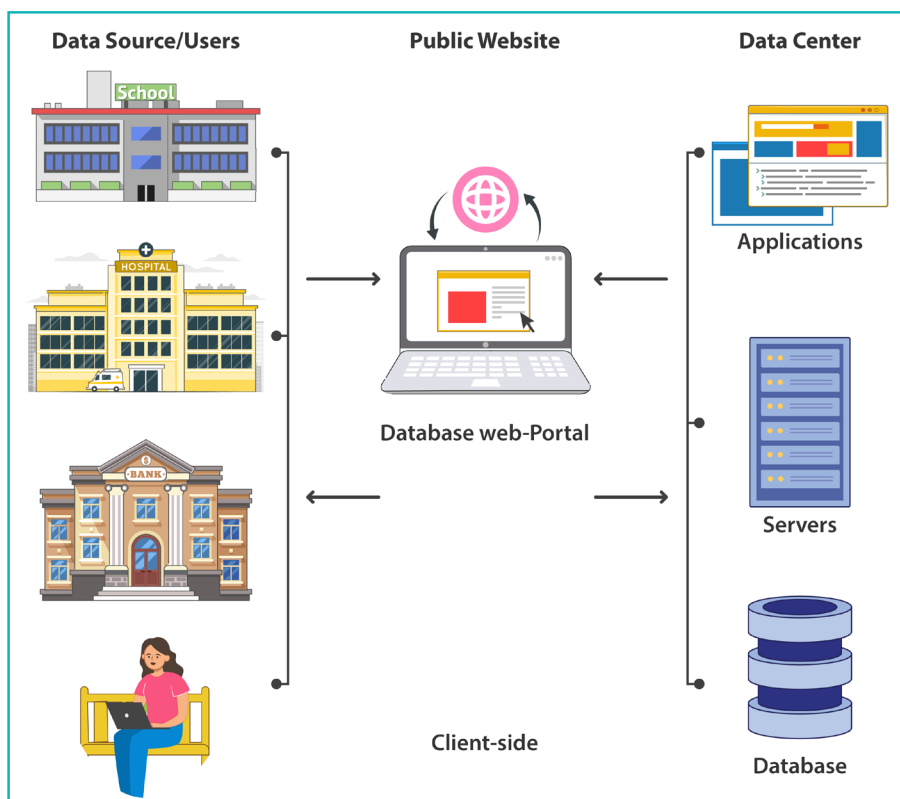
What is Database?

A Database is a structured collection of related data/information. All the data in a database is related to each other; hence, it is called 'organised data'. For example, a school's database should include all student-related information, such as student marks, subjects, subject teachers, grades, etc.

Another example would be an e-commerce application that lists numerous products. The e-commerce application stores product names, product images, prices, size details, and other information. All this data of the numerous products is stored in the database. Databases store data electronically in a computer system, making it easily accessible and manageable.

Storing data in a database offers several advantages, such as:

- Easy to store and extract thousands of stored records within seconds.
- Thousands of users can access the same database from different locations at the same time.
- It also protects our data by requiring login credentials to access the data stored in the database.



The importance of data and databases cannot be overstated in today's digital age. Here is a brief overview of their significance:

- a. Information Storage and Retrieval:** Data is the raw material that drives decision-making and analysis. Databases serve as structured repositories for storing and organizing vast amounts of information, making it easier to access and retrieve data efficiently.
- b. Improved Data Integrity and Security:** Centralized databases provide mechanisms to ensure data integrity and security. With proper access controls and encryption, databases safeguard sensitive information from unauthorized access and data breaches.
- c. Efficiency and Scalability:** Databases are designed to handle large volumes of data and support concurrent access from multiple users. This scalability ensures that applications maintain performance even as data and user loads increase.
- d. Business Intelligence and Reporting:** Data stored in databases can be used to generate reports and business intelligence dashboards. These insights help in identifying trends, patterns, and opportunities for business growth and optimization.
- e. Supporting Applications:** Many applications and software heavily rely on databases for their functionality. From Customer Relationship Management (CRM) systems to e-commerce platforms, databases provide the backbone for data storage and retrieval.
- f. Historical Analysis:** Databases enable the storage of historical data, allowing organizations to perform trend analysis and track changes over time. This historical perspective is essential for understanding past performance and predicting future trends.
- g. Regulatory Compliance:** In regulated industries, databases are crucial for compliance with data retention and audit requirements. They help maintain accurate records and ensure transparency of operations.

We can simply say that data and databases play an integral role in modern society, powering information-driven decision-making, enabling innovation, enhancing efficiency, and ensuring the security and integrity of critical information. As technology advances, its importance is only expected to grow further.

Applications of Database

Databases are utilized in various fields. A few of those are mentioned below:

- a. Telecom:** To store information regarding calls we make and receive, usage of the network, details of customers, and so on.
- b. Banking:** For storing customers' information, day-to-day transactions, loan information of borrowers, and so on.
- c. Railway:** To store information such as ticket allotments, passengers' information, train schedules, and so on.

d. Social Media: Social media applications use the database to store information about registered users, uploaded images and videos, their daily activities, and so on.



Apart from the above-mentioned fields, Database is also used in various other sectors/fields such as Education, Airline, Employee Management, Online Shopping, Manufacturing Units, and so on.

Database Management System (DBMS)

Database Management System (DBMS) is a type of software that allows you to create, access, update, and delete data from a database in an organized way. DBMS provides an end-user interface for easily managing the database by storing the data systematically. Additionally, it offers various other functionalities, such as monitoring performance, searching data, and creating backups for recovery. For instance, imagine a company using a DBMS to store and manage its vast inventory of products. With the help of the DBMS, they can efficiently organize product information, track stock levels, and generate real-time reports to make informed business decisions.

We can say that a Database Management System serves as the backbone for controlling and managing a database. Some of the popularly used DBMSs include MySQL, Microsoft SQL Server, dBase, MongoDB, and more.



Relational Database Management System (RDBMS)

It is a specific type of Database Management System in which data is structured in tables with rows and columns to organize and manage data. **Relational Database Management System (RDBMS)** systems provide a more standardized and structured approach to data storage and retrieval by defining relationships between separate tables. For example, consider an online retail store using an RDBMS to handle its vast customer data. It allows them to create a 'Customers' table with relevant attributes like name, email, address, and order history. Simultaneously, another table called 'Orders' can be established to store order details such as order ID, product, quantity, and total amount. The RDBMS establishes a relationship between these tables by linking customer IDs to their respective orders, making it efficient to track customer orders and analyse buying patterns.

Furthermore, RDBMS utilize SQL (Structured Query Language) as the standard language for managing the data stored in the database. SQL enables users to perform various operations such as querying specific data, updating records, and generating customised reports from the database with ease.

SQL (Structured Query Language)

SQL is the standardised language used for working with a database. Using SQL, we can perform all operations, such as creating tables, inserting, updating, deleting data, etc. on a database. It performs all these operations using a set of pre-defined commands. It is widely used in RDBMS such as MySQL, SQL Server, PostgreSQL, and so on.

Now that we are familiar with the basic terminologies associated with a database, we can begin working with it. In this lesson, we will be using the MySQL RDBMS.

MySQL

MySQL is the most popular and scalable Open-source Relational Database Management System supported by Oracle. It's remarkable speed and ease of use have made it the top choice for managing relational databases in various applications and industries. With a thriving community of developers, MySQL continues to evolve, ensuring its relevance in the ever-changing tech landscape.

Exercise

 Tick mark ☒ the correct answer

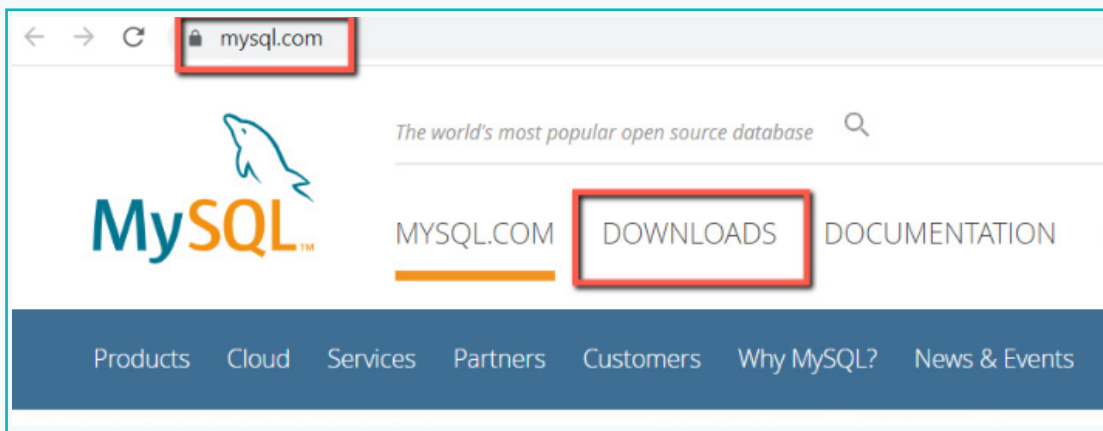
- 1** What does RDBMS stand for?
 - a. Regional Database Management System ☐
 - b. Relational Database Management System ☐
 - c. Relational Data Management System ☐
 - d. Restructure Data Management System ☐

- 2** What is SQL?
 - a. It is a Database Management System. ☐
 - b. It is a Dataset. ☐
 - c. It is the standard language for working with databases. ☐
 - d. It is a RDBMS. ☐

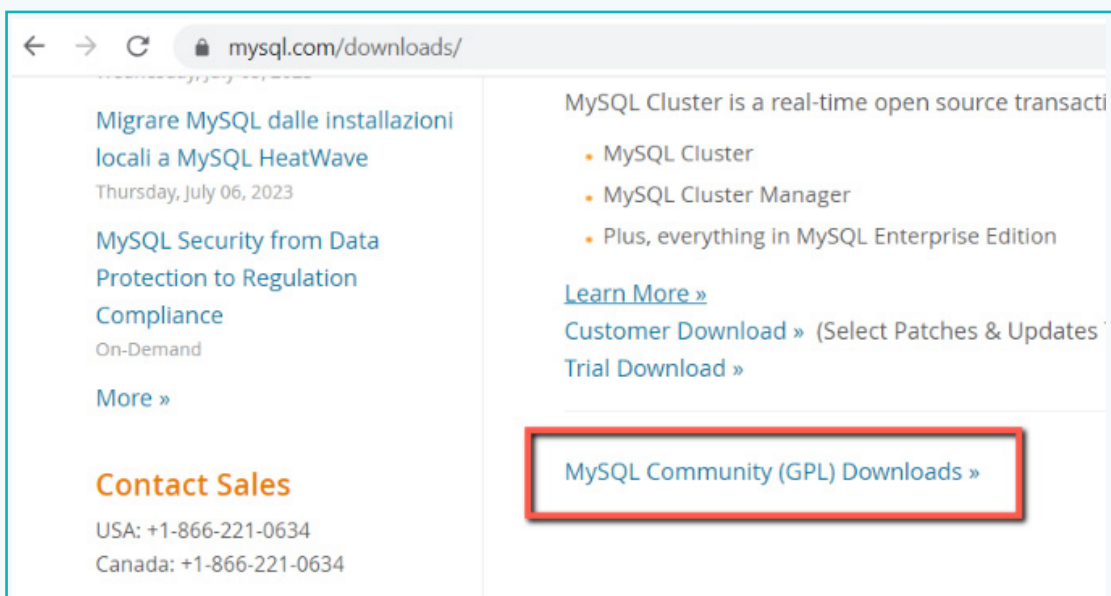
Activity

Installing MySQL RDBMS

- 1 Visit 'mysql.com' and click on 'Downloads'.

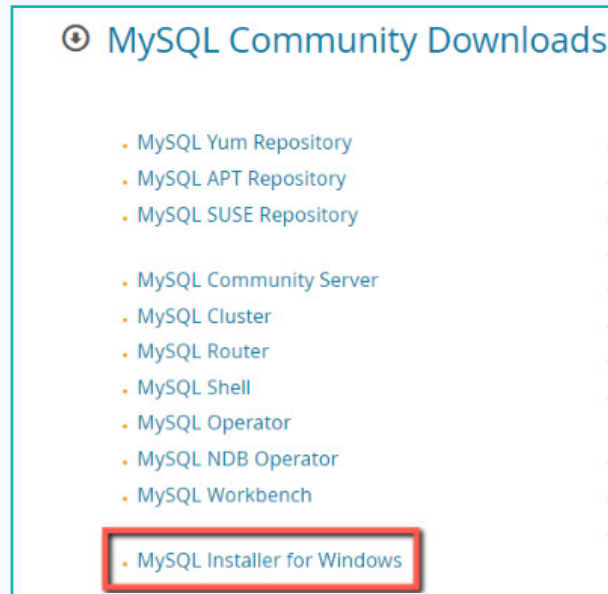


- 2 In the 'Downloads' section, scroll down and click on 'MySQL Community (GPL) Downloads'.

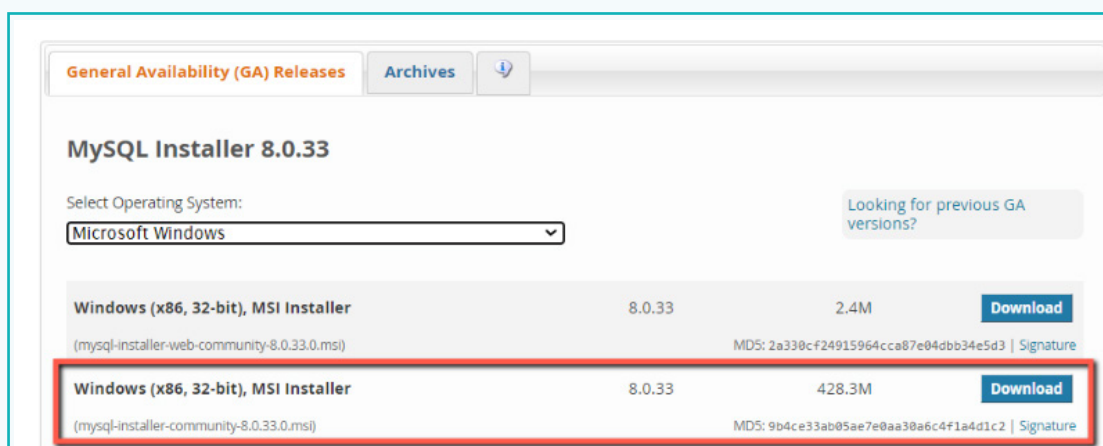


Activity

- 3 This will open up a new page, where we need to click on 'MySQL Installer for Windows'.

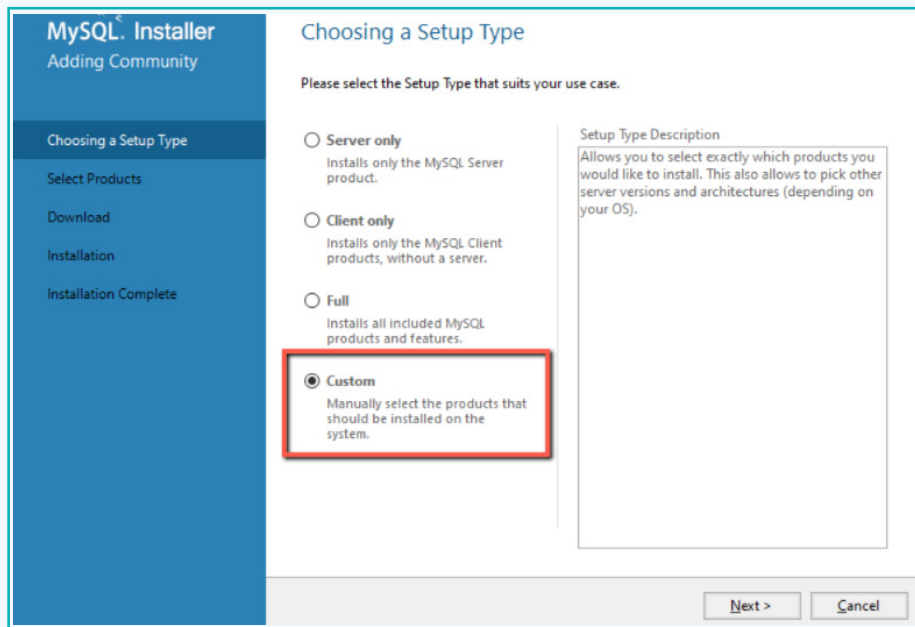


- 4 Finally, we can download the 'MySQL installer' by clicking on the 'Download' button.

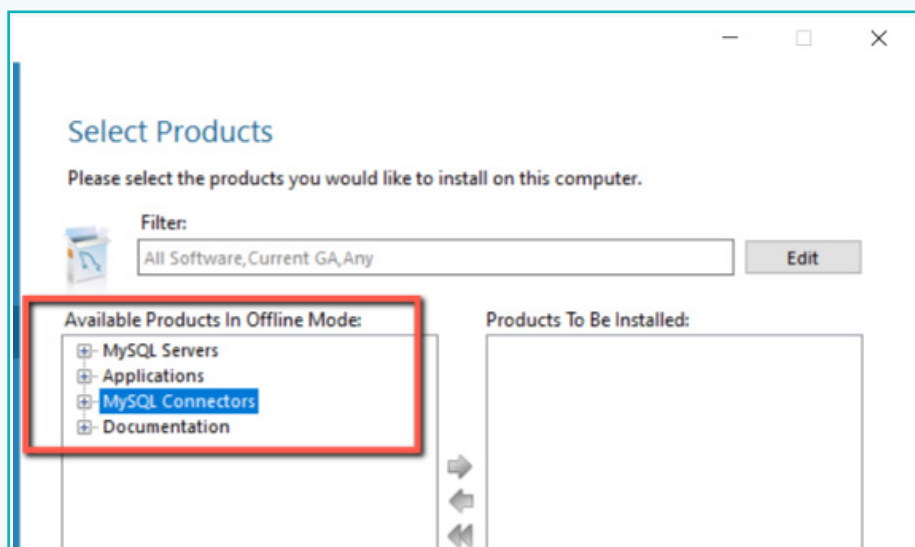


Activity

- 5 Double-click the downloaded file to start with the installation. During the installation process, we will first select the 'Custom' option since we will not require all the components available during installation. Click 'Next'.

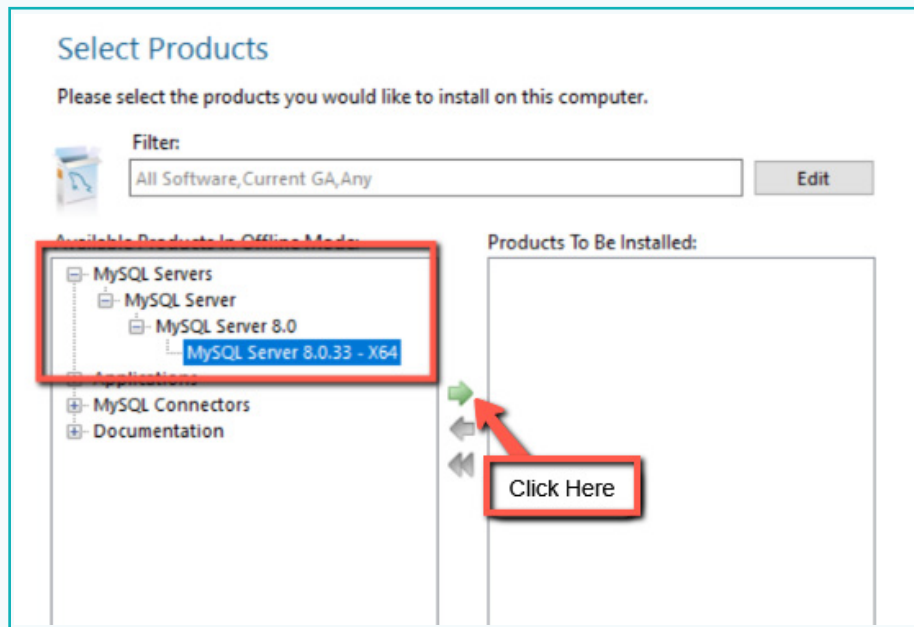


- 6 Next, we will be presented with various products from which we can choose the one to install.

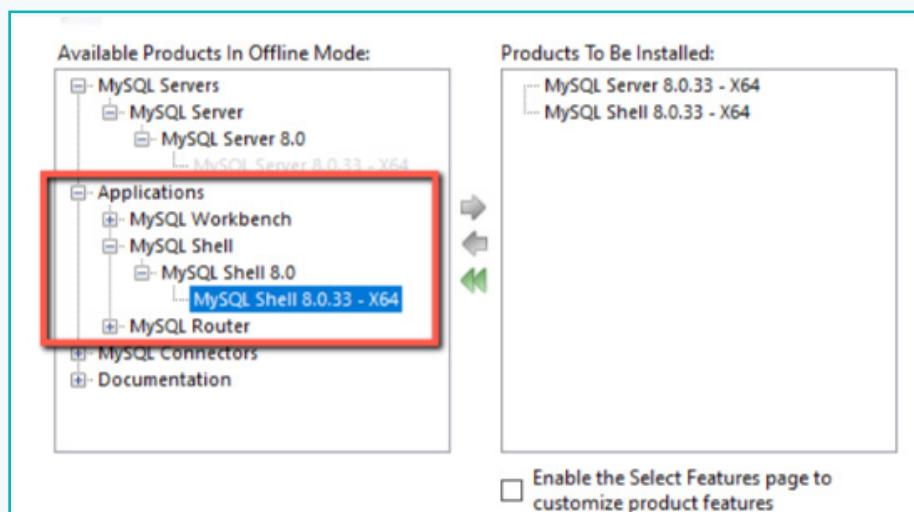


Activity

- 7 Click on the '+' symbol provided by 'MySQL Servers' to expand the menu items. Next, select 'MySQL Server 8.0.33 – X64' and then click on the 'green arrow'. This will move the product to the right window, marking it as the one to be installed.

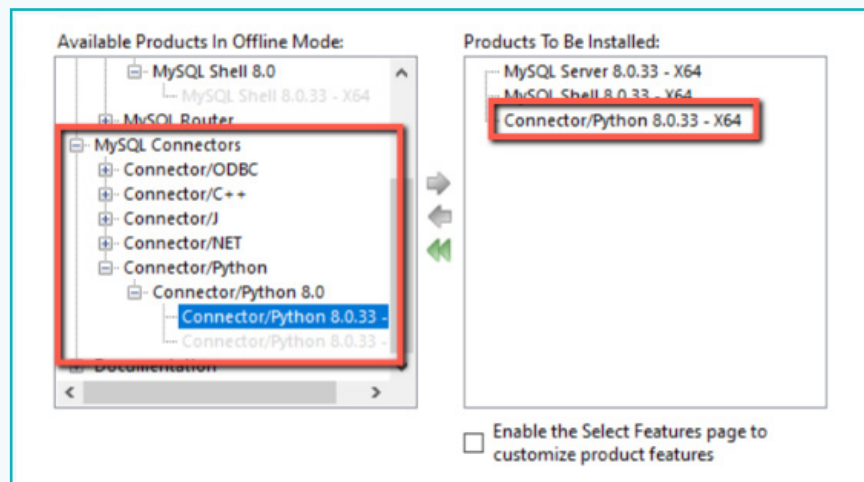


- 8 Next, expand 'Applications', then 'MySQL Shell' and then move the product onto the right window.



Activity

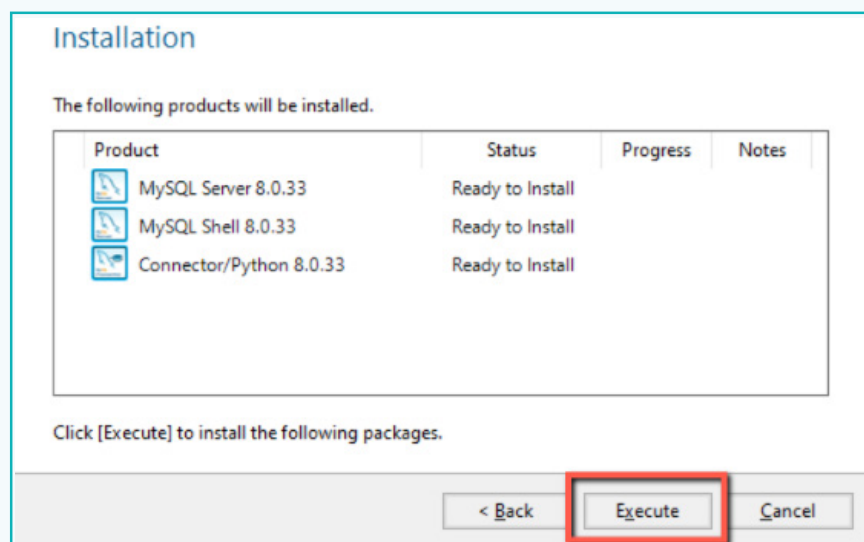
- 9 Finally, expand 'MySQL Connectors', then 'Connector/Python', and move it to the right side as well by clicking on the 'green arrow'.



Note:

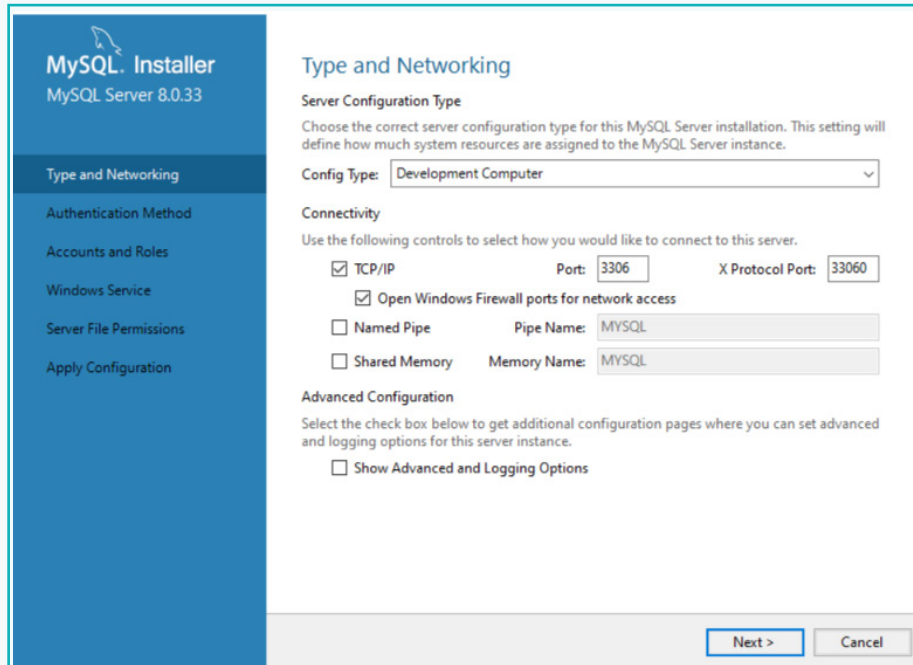
MySQL Connectors are only required if you intend to use MySQL with other programming languages such as Java, Python, .net, and so on.

- 10 Next, click on 'Execute' to install the selected products.



Activity

- 11 Next, you will be presented with a bunch of configuration windows. Keep the default settings.

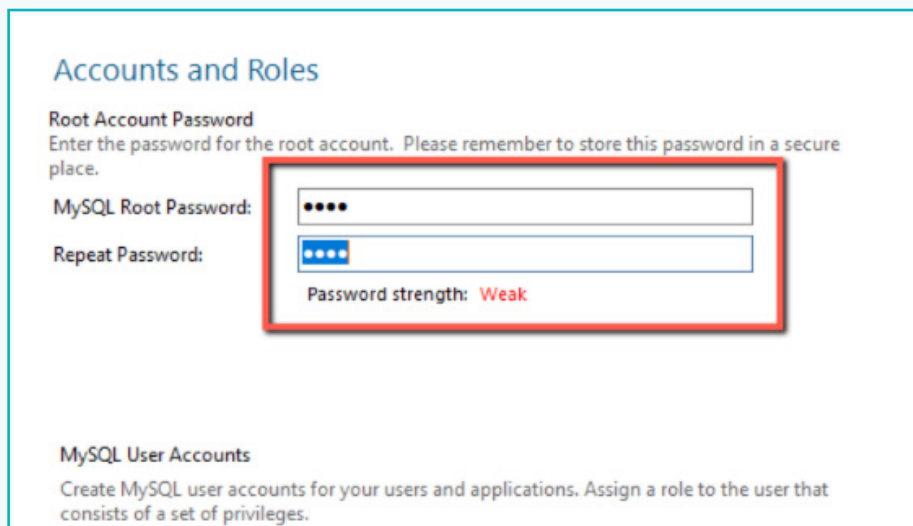


The screenshot shows the 'MySQL Installer' window for 'MySQL Server 8.0.33'. The left sidebar lists configuration steps: 'Type and Networking' (selected), 'Authentication Method', 'Accounts and Roles', 'Windows Service', 'Server File Permissions', and 'Apply Configuration'. The main area is titled 'Type and Networking' and contains the following sections:

- Server Configuration Type:** A dropdown menu set to 'Development Computer'. Below it, a note states: 'Choose the correct server configuration type for this MySQL Server installation. This setting will define how much system resources are assigned to the MySQL Server instance.'
- Connectivity:** A section with the instruction 'Use the following controls to select how you would like to connect to this server.' It includes:
 - ☒ TCP/IP: Port: 3306, X Protocol Port: 33060
 - ☒ Open Windows Firewall ports for network access
 - ☐ Named Pipe: Pipe Name: MYSQL
 - ☐ Shared Memory: Memory Name: MYSQL
- Advanced Configuration:** A section with the instruction 'Select the check box below to get additional configuration pages where you can set advanced and logging options for this server instance.' It includes:
 - ☐ Show Advanced and Logging Options

At the bottom right, there are 'Next >' and 'Cancel' buttons.

- 12 Set a password. This password will be used to access the MySQL terminal and then click the 'Next' button.

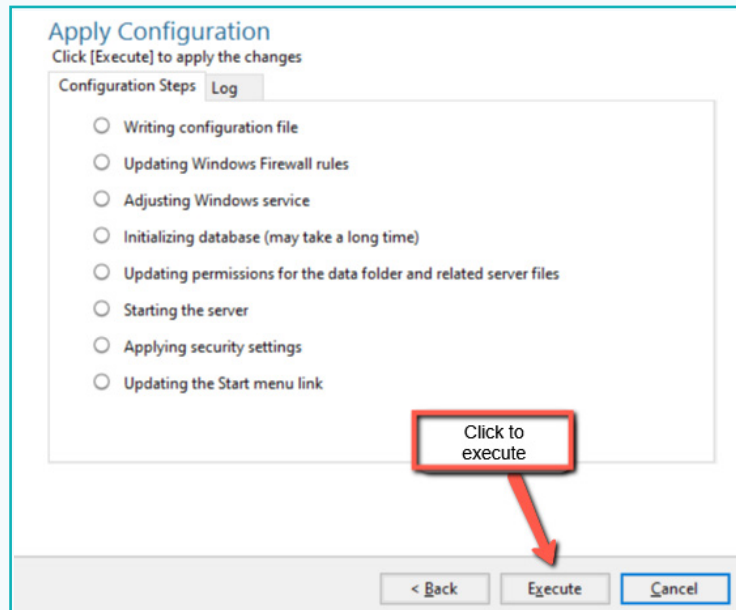


The screenshot shows the 'MySQL Installer' window for 'Accounts and Roles'. The main area is titled 'Accounts and Roles' and contains the following sections:

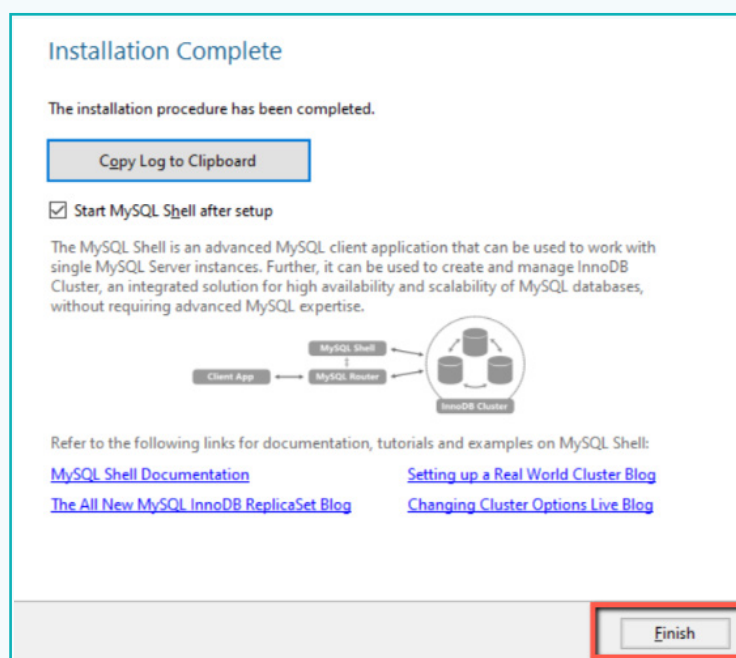
- Root Account Password:** A section with the instruction 'Enter the password for the root account. Please remember to store this password in a secure place.' It includes:
 - 'MySQL Root Password:' followed by a text box with four dots.
 - 'Repeat Password:' followed by a text box with four dots.
 - A red box highlights these two password fields and the 'Password strength: Weak' indicator below them.
- MySQL User Accounts:** A section with the instruction 'Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.'

Activity

- 13 Finally, click on the 'Execute' button to apply the configurations.

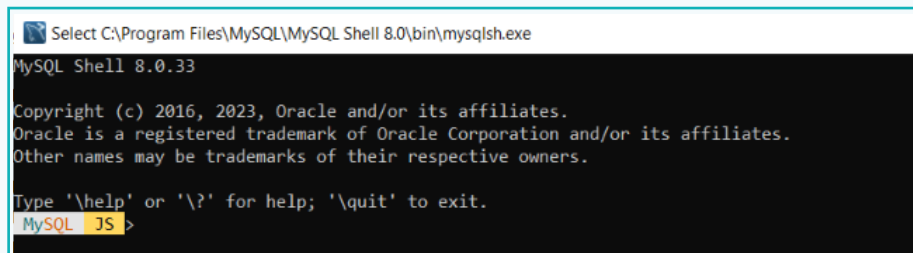


- 14 Click 'Next', then keep the default value for the next settings, and finally click 'Finish'.



Activity

- 15 Once installation is complete, you will be presented with the 'MySQL Shell Terminal'. We are not going to use the Terminal as of now, so simply close the terminal.

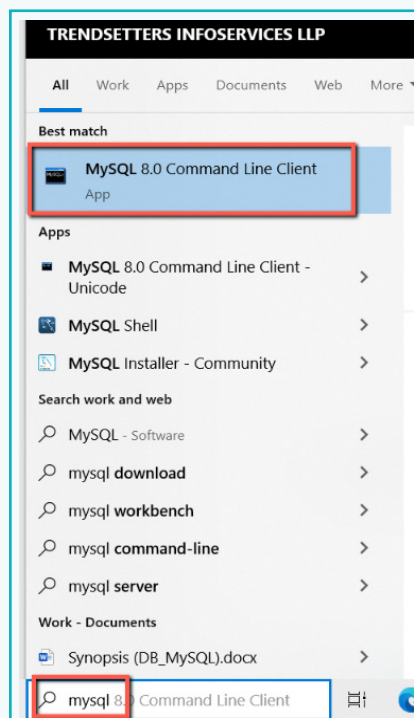


This completes the installation part.

Running the MySQL Command Line Client

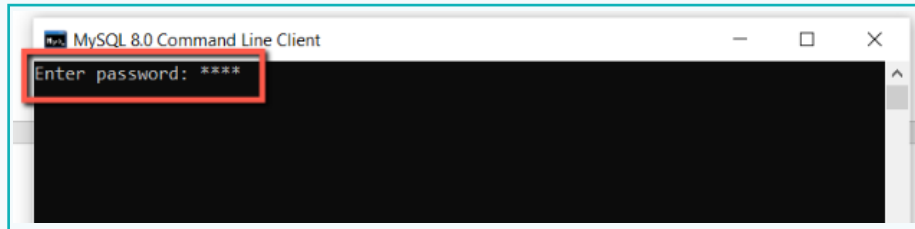
The first step towards creating a database is to access the 'MySQL Command Line Client', which is a Command Line Interface (CLI) provided by MySQL.

- 1 In the 'Start' menu, search for MySQL and open the 'MySQL 8.0 Command Line Client'.

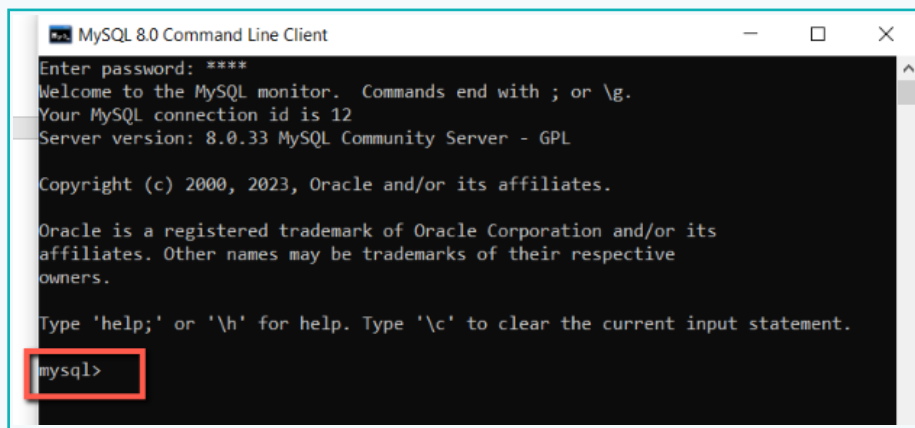


Activity

- 2 In the MySQL Client, enter the password you had set during the installation phase.



- 3 After providing the correct password, you will be able to access the MySQL interface. This is where you will execute the commands for working with a database.



Creating Database

We are going to create a database that will store the details of satellites from different countries. Each country builds its own satellites and launches them into space. Our database will maintain records of the satellites launched by UAE (United Arab Emirates).

The way we work with databases is by executing different SQL Commands on the MySQL Client. Recall that SQL is the language used to communicate with databases.

Note:

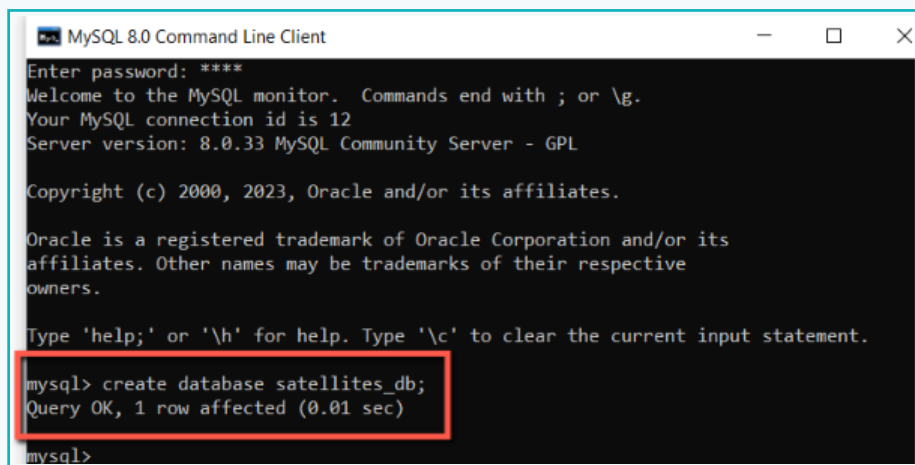
SQL Commands are case-insensitive, which means that the commands can be written in both lower-case and upper-case.

Activity

The command for creating a database is:

'CREATE DATABASE <DATABASE_NAME>;'

- 1 Type the command in the MySQL Client and specify the database name as 'satellites_db'. Press 'Enter' after writing the command.



```
MySQL 8.0 Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.33 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

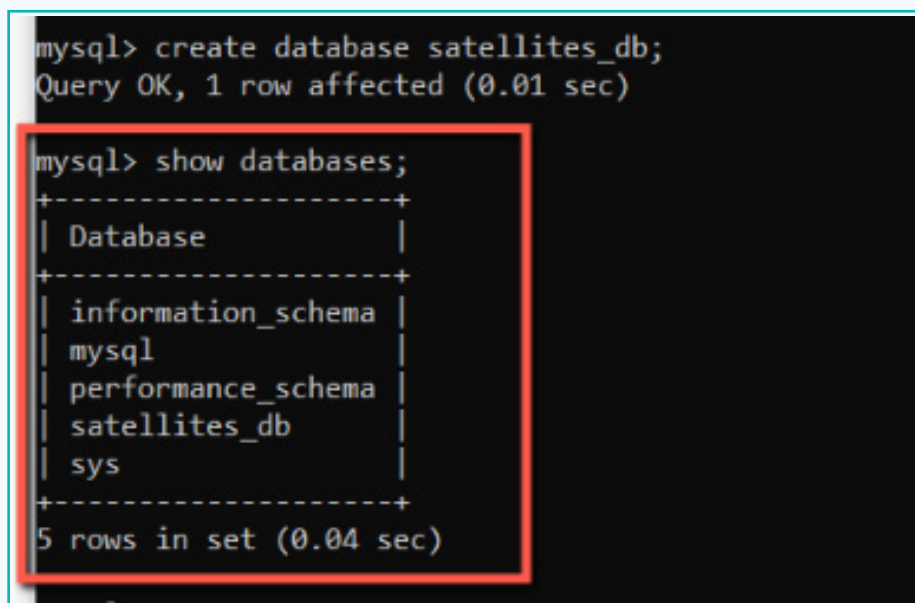
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database satellites_db;
Query OK, 1 row affected (0.01 sec)

mysql>
```

To make sure that the database is created, we can use another command - '**SHOW DATABASES**'. This command displays the names of all the databases.

- 2 Execute the command on the SQL Client.



```
mysql> create database satellites_db;
Query OK, 1 row affected (0.01 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| satellites_db |
| sys |
+-----+
5 rows in set (0.04 sec)
```

Activity

This confirms that our database is created, as the name is displayed after executing the command.

Note that there are also other databases (information schema, mysql, etc.) here. These databases are available by default and are automatically created during the installation process of MySQL.

Tables in a Database

The concept of 'Tables' is fundamental to a database. For storing any data on a database, we need to first create a Table. 'Tables' can be considered as a collection of related data, stored in the form of 'Rows' and 'Columns'. The structure of a table can be compared to an MS Excel spreadsheet, in which data is organised into rows and columns.

'Rows' in a database table represent a single record or an entry. For example, in a 'Student' table, each row would represent or contain information about a particular student. Rows are also referred to as 'Records' or 'Tuples'.

'Columns' in a database table represent different categories of data that we want to store on the table. Each column also has a specific data type (numbers, texts, Boolean, etc.) that describes the type of data that can be stored in that column. For example, a 'Student' table can have various columns such as ID, Name, Address, Phone number, and so on. Columns are also referred to as 'Fields' or 'Attributes'.

Let us see an example of a 'STUDENT' table of a school:

STUDENT_ID	STUDENT_NAME	CITY	PHONE_NO
1	Ryan	Dubai	1234567891
2	Sid	Noida	8523697415
3	Olivia	Sydney	2587413697
4	Thomas	Berlin	15975346824
5	Lily	London	35748964155

Activity

■ Creating Table

The command for creating a table is - '**CREATE TABLE <table_name>**'.

Syntax of the 'CREATE TABLE' command –

```
CREATE TABLE table_name (  
    column_1 datatype,  
    column_2 datatype,  
    column_3 datatype,  
    .....  
    .....  
    column_n datatype  
);
```

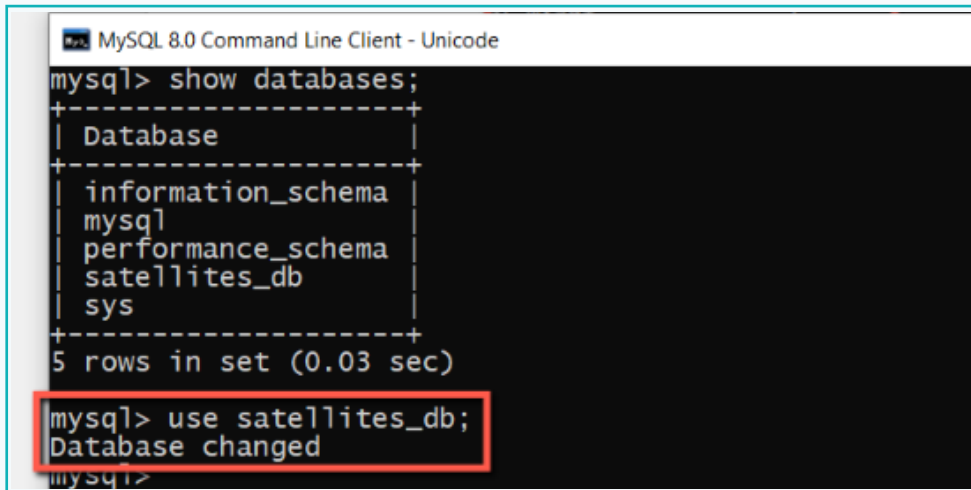
Here, **column_1**, **column_2**, ... **column_n** represents the column or field names, and **datatype** refers to the type of data that will be stored on that column. MySQL supports different datatypes. The common ones are listed below:

- INT, FLOAT for storing numeric values.
- VARCHAR, CHAR for storing strings.
- DATE, TIME for storing dates.
- BOOLEAN for storing Boolean values.

Now, we will create a table 'satellites_uae'. This table will store information about satellites that were launched by UAE.

Activity

- 1 In the MySQL Client, we need to first specify the database where the table will be created. This is done using the 'USE' command and its syntax is – **USE <DATABASE_NAME>;**

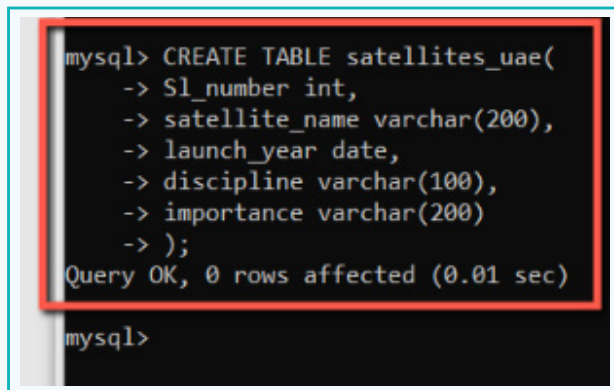


```
MySQL 8.0 Command Line Client - Unicode
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql       |
| performance_schema |
| satellites_db |
| sys        |
+-----+
5 rows in set (0.03 sec)

mysql> use satellites_db;
Database changed
mysql>
```

For each satellite, we are going to store its name, launch year, discipline, and importance. So, based on this, we need to create a table with 4 columns. Additionally, we will also include a serial number for each record we insert into the table, making a total of 5 columns.

- 2 Use the 'CREATE TABLE' command on the MySQL Client to create the table.



```
mysql> CREATE TABLE satellites_uae(
-> Sl_number int,
-> satellite_name varchar(200),
-> launch_year date,
-> discipline varchar(100),
-> importance varchar(200)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Note:

For the 'varchar' datatype, the number in brackets indicates the total number of characters that can be stored for that field or column.

Activity

- 3 Once the table is created, we can use the 'DESC' command to view the structure of the table. So, let us try that out as well.

```
mysql> DESC satellites_uae;
```

Field	Type	Null	Key	Default	Extra
Sl_number	int	YES		NULL	
satellite_name	varchar(200)	YES		NULL	
launch_year	date	YES		NULL	
discipline	varchar(100)	YES		NULL	
importance	varchar(200)	YES		NULL	

```
5 rows in set (0.00 sec)
```

Note:

The command 'DESC <table_name>;' can also be written as 'DESCRIBE <table_name>;'.

Inserting Data into a Table

Now that we have our table created, the next step would be inserting some data into the table. The command is as follows:

INSERT INTO <table_name> VALUES (value_1, value_2...value_n);

The following points should be considered while inserting values into a table:

- 3 The order of providing the values should be the same as the field names.
- 4 String values should be written within double or single quotes, whereas no quotes are required for inserting numeric values.

Now, we will insert data into the 'satellites_uae' table using the 'INSERT' command.

- 5 In the MySQL Client, execute the 'INSERT' command to create the first record on the table.

```
mysql> INSERT INTO satellites_uae VALUES(1, 'DubaiSat-1','2009-07-29','Earth Observation','Remote Sensing, Capturing high resolution earth images for urban planning and disaster management');
Query OK, 1 row affected (0.00 sec)
```

Exercise

■ Fill in the blanks.

- 3 To create a database, the command used is _____.
- 4 To view the structure of a database, the command used is _____.

Activity

■ Viewing Data Stored in a Table

To view the data or records stored in a MySQL table, the 'SELECT' command is used. It has the following syntax –

SELECT * FROM <table_name>;

Here, * is a wildcard operator, which represents all columns in the table.

- 1 In the MySQL Client, execute the 'SELECT' command to retrieve all records from the 'satellites_uae' table.

```
mysql> SELECT * FROM satellites_uae;
+-----+-----+-----+-----+-----+
| Sl_number | satellite_name | launch_year | discipline | importance |
+-----+-----+-----+-----+-----+
| 1 | DubaiSat-1 | 2009-07-29 | Earth Observation | Remote Sensing, Capturing high resolution |
| earth images for urban planning and disaster management |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Activity

- 2 Let us insert two more records into the table.

```
mysql> INSERT INTO satellites_uae VALUES(2, 'KhalifaSat','2018-10-29','Earth Observation','Detailed 3D imagery of Earth');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO satellites_uae VALUES(3, 'Nayif-1','2017-02-15','Education','Nano satellite launched for space technology knowledge development');
Query OK, 1 row affected (0.00 sec)
```

- 3 Display the data using the 'SELECT' command.

```
mysql> SELECT * from satellites_uae;
+-----+-----+-----+-----+-----+
| Sl_number | satellite_name | launch_year | discipline | importance |
+-----+-----+-----+-----+-----+
| 1 | DubaiSat-1 | 2009-07-29 | Earth Observation | Remote Sensing, Capturing high resolution earth images for urban planning and disaster management |
| 2 | KhalifaSat | 2018-10-29 | Earth Observation | Detailed 3D imagery of Earth |
| 3 | Nayif-1 | 2017-02-15 | Education | Nano satellite launched for space technology knowledge development |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

We can also modify the 'SELECT' command to include specific field names. Let us check that.

- 4 Execute the 'SELECT' command by replacing * with "satellite_name, and launch_year" to retrieve only the satellite name and launch year.

```
mysql> SELECT satellite_name, launch_year from satellites_uae;
+-----+-----+
| satellite_name | launch_year |
+-----+-----+
| DubaiSat-1 | 2009-07-29 |
| KhalifaSat | 2018-10-29 |
| Nayif-1 | 2017-02-15 |
+-----+-----+
3 rows in set (0.00 sec)
```

Activity

Updating Records in a Table

MySQL also allows us to modify or update existing records in a table using the 'UPDATE' command. The syntax of which is shown below:

UPDATE <table_name>

SET column1 = value1, column2 = value2, ...

WHERE condition;

To test the command, we will modify the existing value of the discipline field for the Nayif-1 satellite.

- 1 Execute the following 'UPDATE' command for the Nayif-1 satellite record to replace 'Education' with 'Space Education'.

```
mysql> UPDATE satellites_uae
-> SET discipline = 'Space Education'
-> where Sl_number = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- 2 The above command can also be written in a single line, although writing the command in multiple lines improves the readability of the code.

```
mysql> UPDATE satellites_uae SET discipline = 'Space Education' WHERE Sl_number = 3;
```

- 3 Now, if we execute the 'SELECT' command to view the records, we will find the updated data.

```
mysql> SELECT * from satellites_uae;
+-----+-----+-----+-----+-----+
| Sl_number | satellite_name | launch_year | discipline | importance |
+-----+-----+-----+-----+-----+
| 1 | DubaiSat-1 | 2009-07-29 | Earth Observation | Remote Sensing, Capturing high resolution earth images for urban planning and disaster management |
| 2 | KhalifaSat | 2018-10-29 | Earth Observation | Detailed 3D imagery of Earth |
| 3 | Nayif-1 | 2017-02-15 | Space Education | Nano satellite launched for space technology knowledge development |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Activity

Deleting Records from a Table

To delete records from MySQL, it provides the 'DELETE' command. It allows us to remove one or more records from the table based on specified conditions. The syntax for the 'DELETE' command –

DELETE FROM table_name WHERE condition;

- 1 Let us delete the last record from the table using the 'DELETE' command.

```
mysql> DELETE FROM satellites_uae WHERE Sl_number = 3;  
Query OK, 1 row affected (0.00 sec)
```

- 2 Execute the 'SELECT' command to verify the 'DELETE' operation.

```
mysql> SELECT * from satellites_uae;  
+-----+-----+-----+-----+-----+  
| Sl_number | satellite_name | launch_year | discipline | importance |  
+-----+-----+-----+-----+-----+  
| 1 | DubaiSat-1 | 2009-07-29 | Earth Observation | Remote Sensing, Capturing high  
resolution earth images for urban planning and disaster management |  
| 2 | KhalifaSat | 2018-10-29 | Earth Observation | Detailed 3D imagery of Earth |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

As can be seen, the third record is no longer present on the Table.

Exercise

State if the statements are True or False

5 To view the records stored in a database table, we use the 'SELECT' command.

 _____

6 To remove records from a database table, we use the 'REMOVE' command.

 _____

In this lesson, we explored the fundamentals of databases, focusing on MySQL as our Relational Database Management System (RDBMS) of choice. We learned that a database is a structured collection of related data that is essential for efficiently managing and organizing information. Databases offer numerous advantages, including easy data storage and retrieval, multi-user access, and data security. They play a pivotal role in data-driven decision-making, business intelligence, and supporting various applications. We discussed how to install MySQL, create a database, and create, view, update, and delete records in a table. Understanding databases and their management is crucial in today's digital age, enabling us to harness the power of data for informed decision-making, innovation, and organisational growth.

With MySQL as a foundational RDBMS, businesses benefit from streamlined data management, informed decision-making, and enhanced efficiency, enabling growth and innovation in the digital era.



Tech Flash

- **NoSQL** : These are non-tabular, schema-less databases that store data as Documents and Collections.
- **MongoDB** : It is a document-oriented, NoSQL database program and can be used as an alternative to traditional database programs. They are commonly used to manage large amounts of distributed data.